

LAPORAN UTS PRAKTIKUM INDIVIDU: COMMUNICATION PROTOCOL

(CASE 2 - PRODUCT/INVENTORY API)

Disusun Oleh:

Nama : Fadli Ismail
NIM : 25120500009
Kelas : ComP2 - Profesional
Judul Case : Case 2 - Product/Inventory API
Link GitHub : <https://github.com/Fdlisml/communication-protocol-uts>

1. RINGKASAN CASE DAN SKENARIO

Laporan ini mendokumentasikan pengujian teknis terhadap Product/Inventory API untuk memvalidasi kepatuhan arsitektur terhadap standar HTTP/REST. Tujuan utama dari pengujian ini adalah memastikan fungsionalitas manajemen inventaris—mencakup pengambilan daftar produk, inspeksi detail resource, penambahan data, dan pembaruan stok—beroperasi secara presisi sesuai protokol komunikasi yang ditetapkan. Dalam skenario ini, client berinteraksi dengan sistem inventaris menggunakan metode GET, POST, dan PATCH untuk mensimulasikan siklus hidup data produk dalam katalog digital. Eksekusi request dilakukan menggunakan **Postman**, dokumentasi versi dikelola melalui **GitHub**, dan pertukaran data menggunakan format **JSON** sebagai standar encoding antar entitas sistem.

2. DESAIN REQUEST

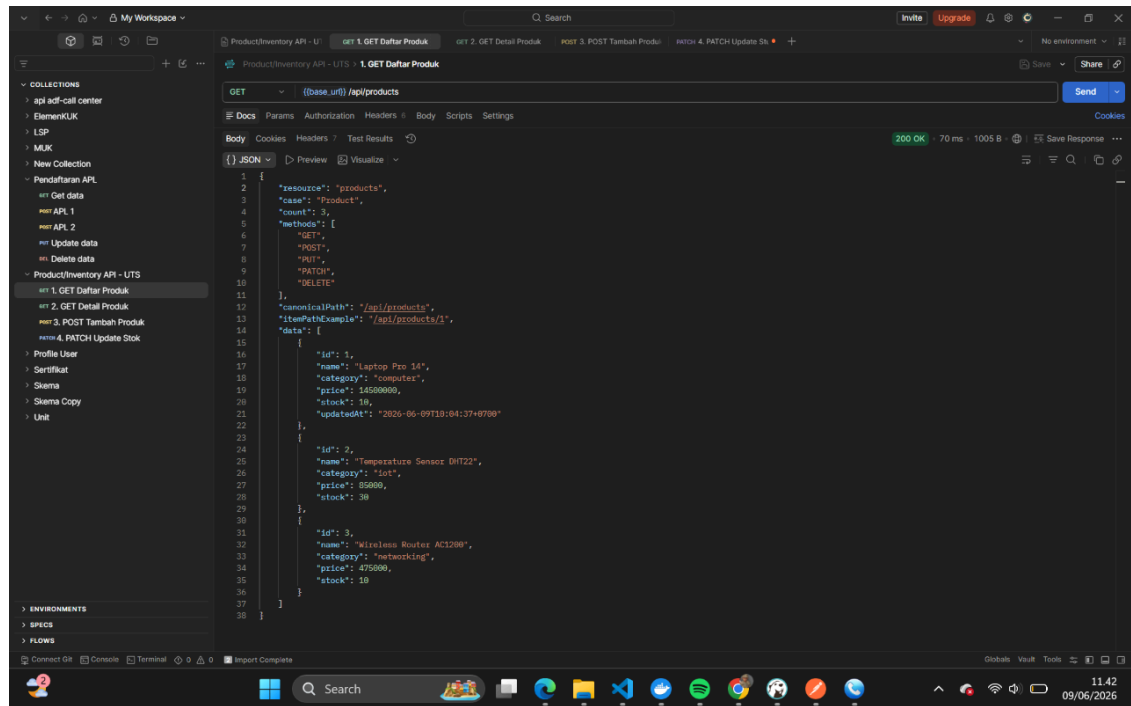
Rencana pengujian dirancang secara sistematis untuk memverifikasi fungsionalitas CRUD (Create, Read, Update) pada resource produk. Berikut adalah tabel desain request yang digunakan dalam simulasi:

No	Method	Endpoint	Tujuan	Expected Response Code
1.	GET	/api/products	Mendapatkan daftar semua produk	200 OK
2.	GET	/api/products/:id	Mendapatkan detail produk ID yang diinput	200 OK
3.	POST	/api/products	Menambah produk baru ke database	201 Created
4.	PATCH	/api/products/:id	Update stok produk ID yang diinput secara parsial	200 OK

Metode **PATCH** diimplementasikan untuk pembaruan stok guna mencapai efisiensi komunikasi data. Secara semantik, PATCH digunakan untuk modifikasi parsial pada field stock tanpa harus mengirimkan seluruh payload resource, berbeda dengan metode PUT yang bersifat idempoten untuk penggantian resource secara utuh.

3. EVIDENCE REQUEST GET

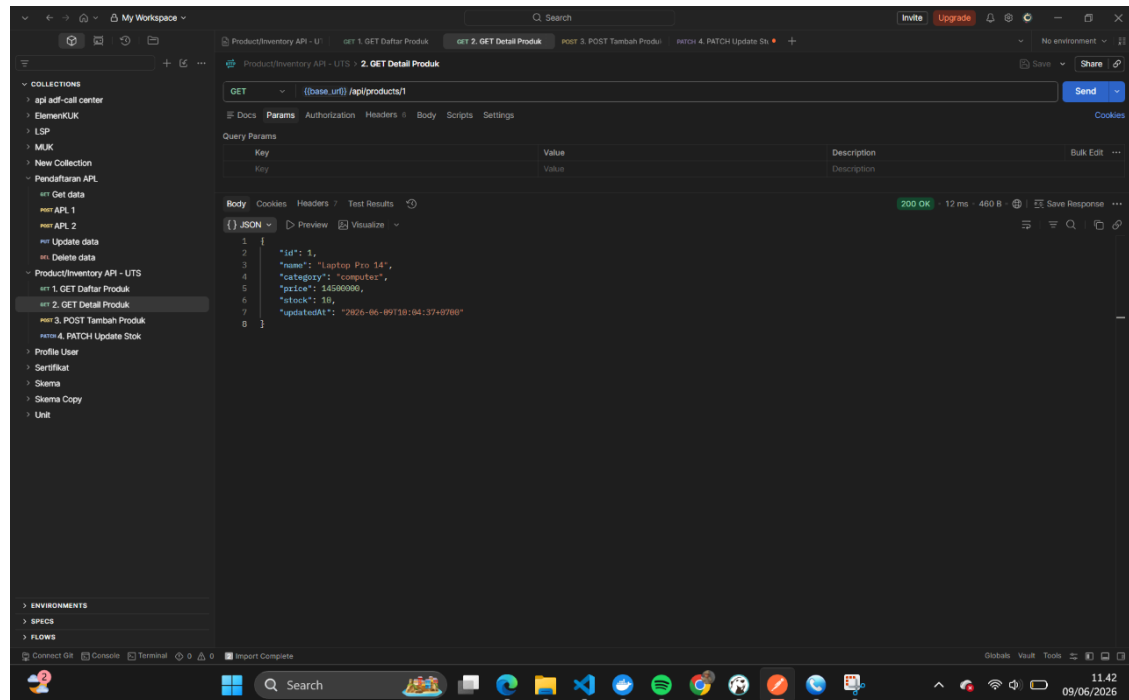
3.1. Request 1: GET Daftar Produk



Referensikan bukti eksekusi pada file get-01.png.

- **Method:** GET
- **URL:** `{{base_url}}/api/products`
- **Status Code:** 200 OK
Request ini memvalidasi kemampuan server dalam mengagregasi seluruh resource produk yang tersedia. Berdasarkan metadata pada response body, jumlah entitas yang diterima teridentifikasi pada field count: 3.

3.2. Request 2: GET Detail Produk

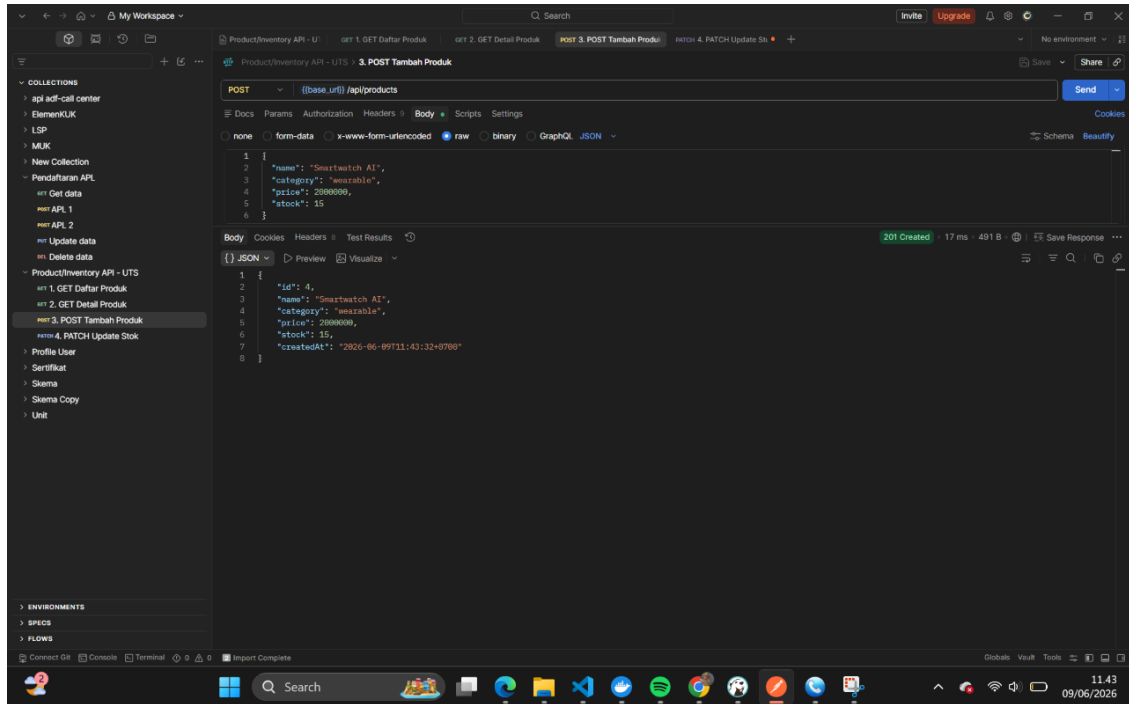


Referensikan bukti eksekusi pada file get-02.png.

- **Method:** GET
- **URL:** {{base_url}}/api/products/1
- **Status Code:** 200 OKPengujian ini bertujuan melakukan *fetch* data spesifik untuk ID 1. Server mengembalikan resource "Laptop Pro 14" dengan raw JSON value untuk harga sebesar 14500000 (representasi desimal: 14.500.000).

4. EVIDENCE REQUEST POST & PATCH

4.1. Request 3: POST Tambah Produk

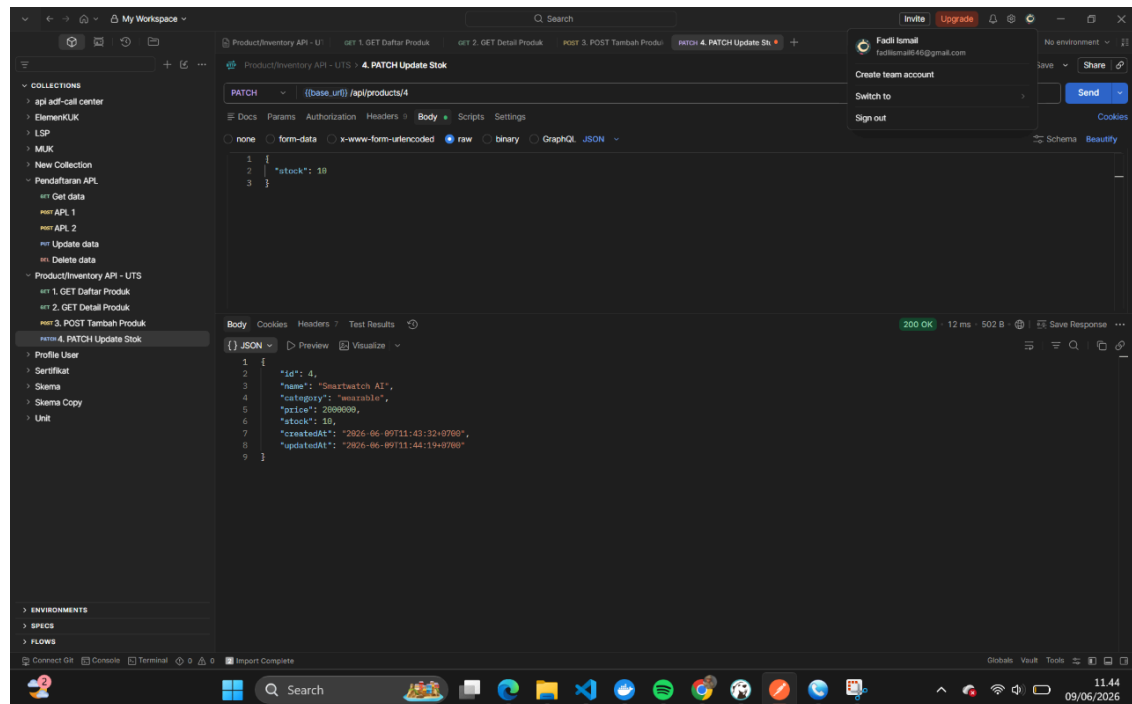


Referensikan bukti eksekusi pada file post-01.png.

- **Method:** POST
- **URL:** {{base_url}}/api/products
- **Status Code:** 201 Created Server berhasil melakukan persistensi data untuk resource baru dengan payload JSON sebagai berikut:

```
{
  "name": "Smartwatch AI",
  "category": "wearable",
  "price": 2000000,
  "stock": 15
}
```

4.2. Request 4: PATCH Update Stok



Referensikan bukti eksekusi pada file post-02.png.

- **Method:** PATCH
- **URL:** `{{base_url}}/api/products/4`
- **Status Code:** 200 OK Request ini berhasil mengubah nilai field stock menjadi 10 pada produk ID 4. Keberhasilan transaksi ini dibuktikan dengan pembaruan otomatis pada metadata waktu, di mana field `updatedAt` mencatat timestamp 2026-06-09T11:44:19+0700, menjamin integritas audit data pada level database.

5. ANALYSIS RESPONSE API

Sebagai spesialis protokol, saya melakukan dekonstruksi terhadap komponen response yang diterima:

- **Status Codes:**
- **200 OK:** Menunjukkan suksesnya query (GET) dan modifikasi resource (PATCH). Secara arsitektural, pengembang juga dapat menggunakan *204 No Content* untuk PATCH jika tidak ada body yang dikembalikan, namun penggunaan 200 OK di sini memberikan feedback data yang telah diperbarui.
- **201 Created:** Mengonfirmasi bahwa request POST telah berhasil menciptakan resource baru dengan URI yang unik.
- **Headers:** Identifikasi header menunjukkan penggunaan `Content-Type: application/json` yang mendefinisikan format payload. Berdasarkan evidence, header standar lain seperti `Date`, `Content-Length`, dan `Server` juga hadir untuk mengelola siklus hidup koneksi HTTP.

- **Response Body (HATEOAS):** Struktur body pada endpoint daftar produk menerapkan gaya HATEOAS (Hypermedia as the Engine of Application State). Metadata seperti resource, methods, dan canonicalPath menyediakan navigasi dinamis bagi client untuk memahami kapabilitas API tanpa dokumentasi eksternal yang statis.

6. ANALYSIS DATA ENCODING (JSON)

Analisis mendalam terhadap struktur JSON pada evidence mengungkapkan skema data berikut:

- **Type Data:**
- **Integer:** Digunakan untuk nilai numerik presisi seperti id, price, dan stock.
- **String:** Digunakan untuk deskripsi tekstual (name, category) dan representasi waktu.
- **Array:** Teridentifikasi dua variasi penggunaan array: methods merupakan **array of strings** (kumpulan metode HTTP), sedangkan data merupakan **array of objects** (kumpulan entitas produk).
- **Struktur Nested Object:** Pada endpoint /api/products, implementasi *nested object* terlihat di mana objek root membungkus metadata navigasi, dan data produk dienkapsulasi di dalam array data.
- **Format Timestamp:** Field updatedAt dan createdAt menggunakan standar **ISO 8601**. Penggunaan offset +0700 secara spesifik mengidentifikasi zona waktu **Western Indonesia Time (WIB)**, yang krusial untuk sinkronisasi data pada sistem terdistribusi.

7. KESIMPULAN

Praktikum Case 2 (Product/Inventory API) menunjukkan bahwa API telah memenuhi seluruh kriteria operasional CRUD dengan kepatuhan penuh terhadap semantik REST. **Kendala:** Tidak ditemukan kendala teknis yang signifikan; seluruh endpoint memberikan respon yang konsisten dengan desain request. **Pembelajaran Utama:**

1. **Semantik HTTP:** Pemilihan metode (GET vs POST vs PATCH) sangat menentukan efisiensi payload dan kejelasan tujuan instruksi pada server.
2. **Metadata & Headers:** Komponen seperti Content-Type dan metadata HATEOAS sangat krusial dalam membangun API yang *self-describing*.
3. **Standarisasi Data:** Penggunaan standar ISO 8601 untuk waktu dan JSON sebagai format encoding memastikan interoperabilitas data yang tinggi antar platform.